



Module 22

Introduction to Metric Driven Verification and Planning

cā dence[®]

This training module introduces you to Metric driven verification or MDV and the concept of verification planning.

Objectives

In this module, you will:

- Define Metric Driven Verification (MDV)
- Define the planning aspect of Verification



In this submodule, you will:

- Define MDV.
- Define the planning aspect of vManager.
- Identify metrics.
- Map metrics to your vPlan.

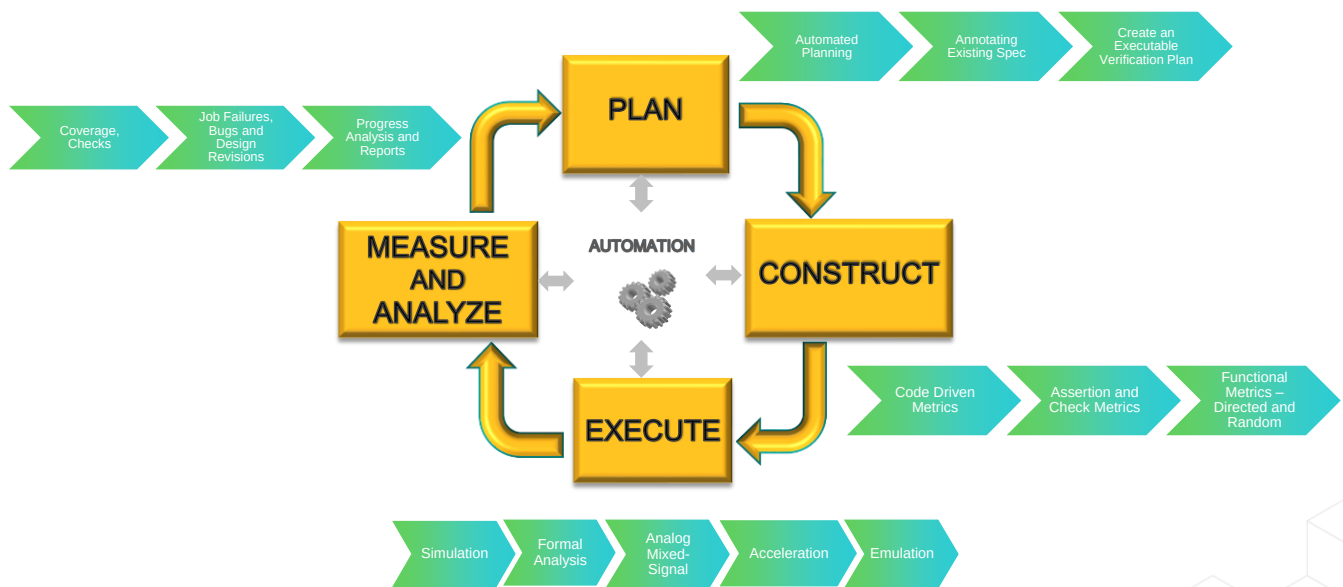
What Is Metric Driven Verification or MDV?

- MDV is a powerful layer of methodology that sits above the Verification Testbench environment, which provides guidelines and tools for using metrics and automation to maximize the benefits of the verification testbench.
- It is a data-driven, decision-based flow that improves the Predictability, Productivity and Quality of the verification effort.
- MDV is a **closed-loop process** with the following stages:
 - **PLAN** an Executable Verification Plan
 - **CONSTRUCT** a testbench and other verification infrastructure using Universal Verification Methodology (UVM)
 - **EXECUTE** the regression on various engines such as Simulation, Formal Verification, Emulation, etc.
 - **MEASURE AND ANALYZE** the results to see what else is left to be done to finish



This page does not contain notes.

MDV: A Closed-Loop Process



4

© Cadence Design Systems, Inc. All rights reserved.

cadence

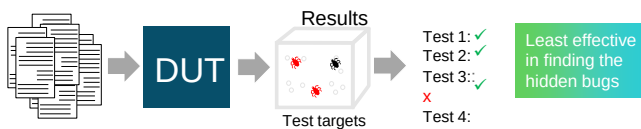
- **Planning:** The MDV solution utilizes the most powerful specific-purpose verification plan authoring software, the vPlanner tool, which is an integral part of the Cadence vManager™ solution. The MDV flow starts with automated planning, either annotating existing specifications with verification intent or creating an executable verification plan using team member inputs captured in a spreadsheet.
- **Construction:** MDV spans from simply directed test methodologies through to register-transfer level (RTL)-based coverage-driven and software-driven approaches. For RTL verification, the Universal Verification Methodology (UVM) standard has been specifically designed to provide powerful constrained random testing. The power of constrained random stimulus is that vectors are automatically generated without having verification engineers spell out each set of input vectors manually. With the constrained random stimulus come constrained random changes in state. In this environment, it is imperative that the verification team is able to specify which states have been explored and that for those explored states, the correct response to stimulus vectors is observed. For this reason, functional coverage is utilized. Functional coverage is defined for and captures scenarios that are important for the verification environment to observe and capture.
- **Execution:** Verification simulation, formal analysis, analog mixed-signal, acceleration, and emulation execution engines are dispatched and the convergence of results from multiple verification engines is managed within the Cadence vManager environment.
- **Measurement/Analysis:** MDV is used to measure real-time progress with on-demand information as well as to determine when high-quality verification closure is achieved. Coverage, checks, and assertions provide the verification-specific metrics used to determine closure. Verification job failures, bugs, and design revisions all provide insight into the actual status of a project. Progress analysis and reports help the verification team make adjustments to their resource allocation (people and tools) to reach closure more efficiently and measure closure more accurately.

What Are the Other Prevalent Different Methods of Verification?

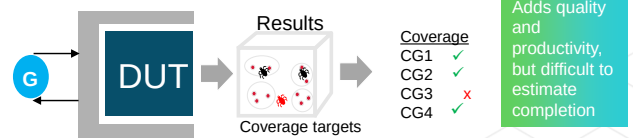
There are other prevalent Verification Methods in the industry, such as the Traditional Directed Testing Approach and Coverage Driven Verification (CDV).

- **Traditional Directed Testing Verification** involves a planning process that usually documents what test cases need to be written, but the underlying reason for the said test cases should be due to design analysis.
- Manual testing is tedious and labor-prone.
- **Coverage-Driven Verification** methodology, or CDV, has been proven to be able to identify bugs faster.
- It can produce huge amounts of coverage data, which can be overwhelming when trying to analyze which design features belong to which coverage points. This overwhelming amount of data is why coverage-driven has many limitations that affect its usability and scalability.

Directed Tests Driven



Coverage Driven



5

© Cadence Design Systems, Inc. All rights reserved.

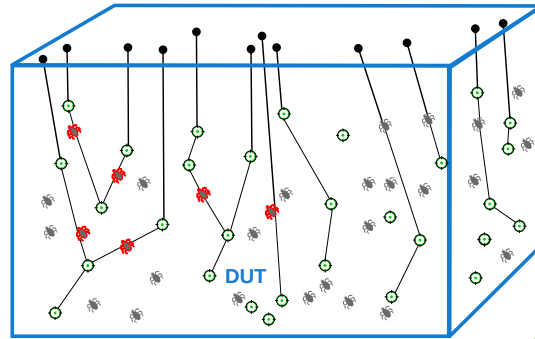
cadence

The plan-based verification method, MDV, is the most successfully adopted method of verification as compared to the others. Let us see why. And then list the benefits of using the MDV Method.

Traditional Approach: Directed Testing

Verification Engineer:

- Defines DUT states to be tested based on specified behavior and corner cases.
- Writes directed test to visit and verify each item in the test plan.



Significant manual effort to write all the tests.



Work required to verify each goal was reached.



Poor coverage of non-goal scenarios ... especially the cases that you didn't "think of."

Redo if design changes



- Traditional Verification Planning process usually indicates what test cases need to be written, but the underlying reason for the said test cases should be due to design analysis. Usually, the classical test plan will then just document what directed tests need to be written. The test plan should be the set of goals to achieve, but most plans document the means by which to achieve those goals.
- Since manual testing is tedious and labor prone, the typical method of copy-pasting snippets in test cases results in relatively low variability from one test case to the next.

Coverage-Driven Verification

Verification Engineer:

- Defines DUT states to be tested based on specified behavior and corner cases.
- Focus moves to reach goal areas (versus execution of test lists).

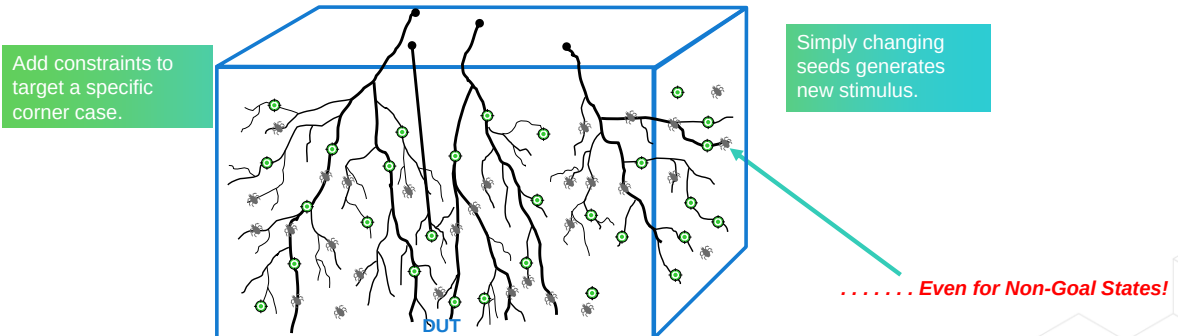
Constrained random stimulus generation explores goal areas (and beyond).

Coverage reports which *goals* have been exercised and which need attention.

Self-Checking ensures proper DUT response.

Automation:

- Constrained random stimulus accelerates hitting coverage goals and exposing bugs.
- Coverage and checking results indicate the effectiveness of each simulation.
- Enables many parallel runs to contribute in order to scale the verification effort.



7

© Cadence Design Systems, Inc. All rights reserved.

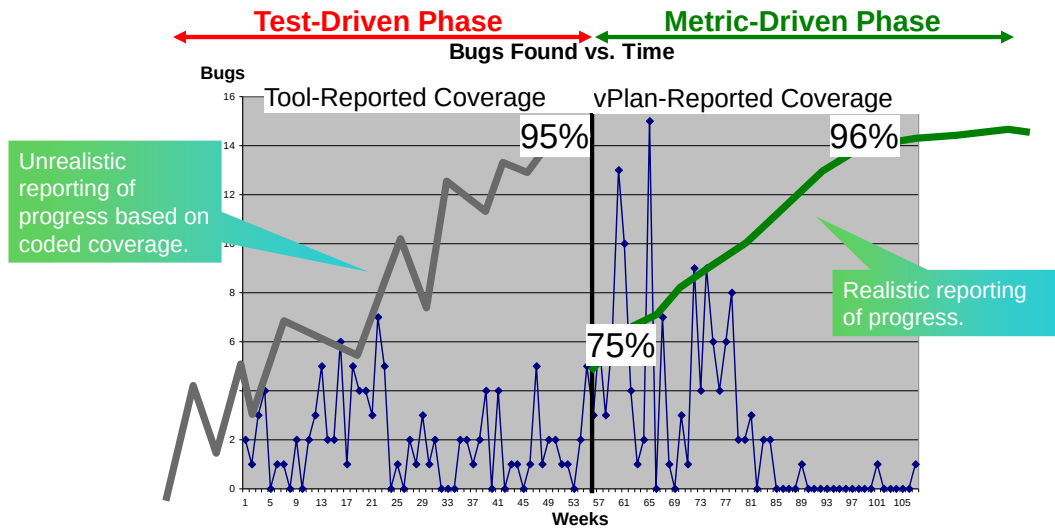
cadence

Coverage-driven verification methodology or CDV has been proven to be able to identify bugs faster.

However, it is difficult to see what has been actually tested because the testbench is, by definition, randomized. Coverage driven is added to visibly see what is tested and what is not during this randomization process. While it is relatively easy to add functional coverage, it can produce huge amounts of coverage data, which can be overwhelming when trying to analyze which design features belong to which coverage points. This overwhelming amount of data is why coverage driven has many limitations that affect its usability and scalability.

Note: The limitations of CDV have been explained in detail in our new ES (Education Services) course, *Foundations of Metric Driven Verification*.

Tool-Reported Coverage Versus Plan-Based Coverage

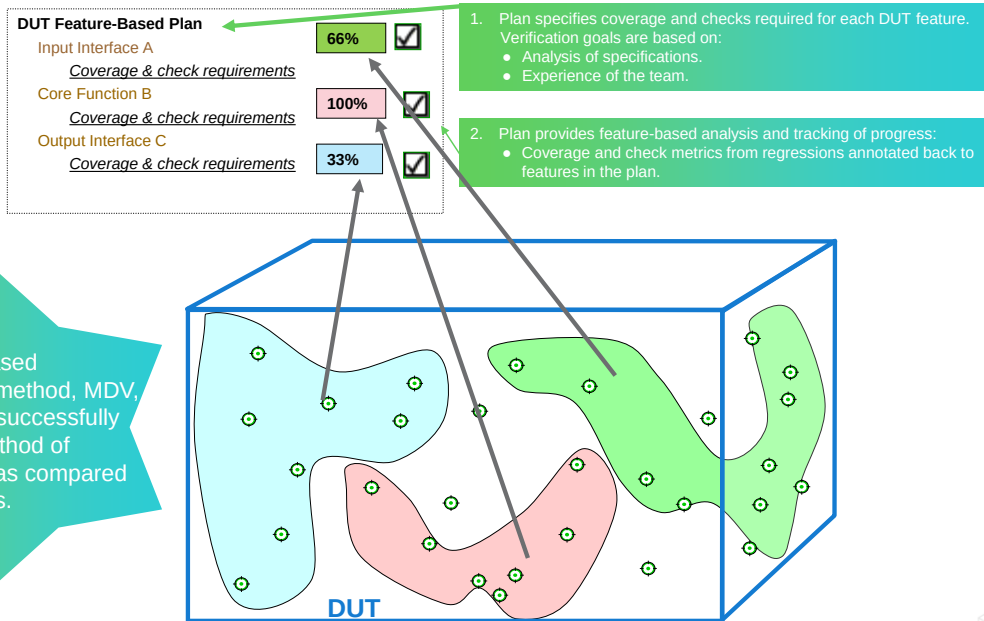


8 © Cadence Design Systems, Inc. All rights reserved.



- Here, we can see the problem of accurately estimating the true completion of the verification due to the lack of a vPlan.
- Before the CDV stage started, we estimated the functional coverage at 95% based on coverage reported by Specman®. As we formalized the vPlan for CDV, we were surprised to see that the actual coverage is only 75%.
- The difference is due to the fact that the normal flat coverage model cannot incorporate the complexity of different levels of importance, while the vPlan makes it possible, thus resulting in a more realistic result.

Planning Is Essential: Defines “What” to Include in CDV Environments

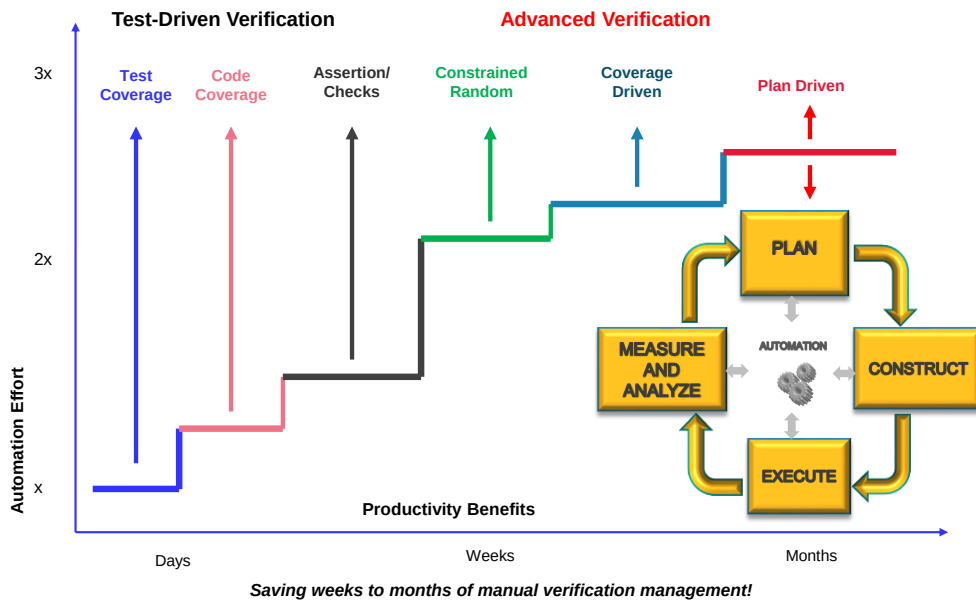


9 © Cadence Design Systems, Inc. All rights reserved.

cadence

- We refer to the “what” as the “features” or “goals” that must be verified – the design intention. When planning a project, you should not presume the underlying tool by which a feature is verified. It will most likely come from several sources, with the results represented by various forms of coverage. Luckily, this top-down approach is not the only way to begin, and starting bottom up, with automation of the test plan, provides instant value in and of itself.
- A verification plan should capture the goals of the verification process in a way that the signoff criteria and milestones are clearly identified. An example of such a milestone is RTL code freeze, which a team might define as the point when 75% of all features are covered. In addition to capturing goals upfront, a verification plan should be executable.
- **Planning answers “what” you need in the verification environment – stimulus, coverage, and checks.**

MDV Productivity Benefits

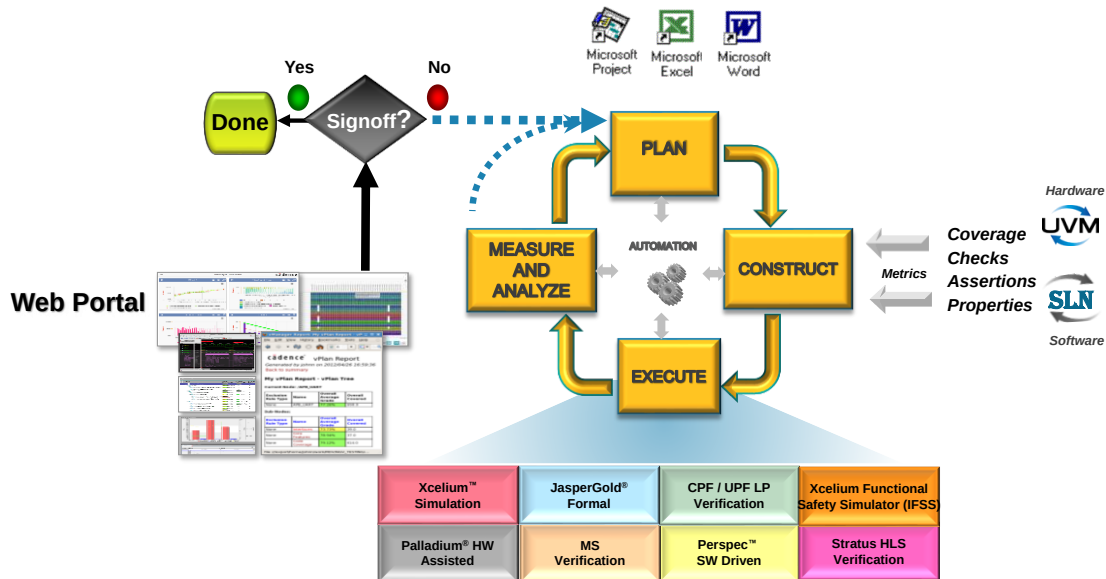


10 © Cadence Design Systems, Inc. All rights reserved.

cadence

- This is a very interesting graph that shows the practicality and benefits of MDV. The metrics in MDV can start as simple “test case coverage” and extend to plan-based closure of design features using the notions of an executable verification plan. As one would expect, increasing investment in automation provides equally increasing productivity benefits from the perspective of time saved or product quality.
- **Test-driven verification** is RTL based and design-centric. It is a very automatic and unobtrusive way to collect test coverage by measuring which tests have passed and which have failed and performing failure analysis.
- Next, adding **code coverage** is as simple as turning on a switch, but now you have visibility to see which parts of the code have been exercised, improving the overall quality of the Verilog or VHDL design.
- Finally, adding assertion-based functional testing with PSL or SVA is a natural fit for additional testing of functionality.
- **Advanced RTL verification** techniques combine constrained random coverage and plan-based techniques to the RTL verification effort. Constrained random testing has proven to be able to identify more bugs faster. However, it is difficult to see what has been tested because the testbench is randomized. While it is relatively easy to add functional coverage, it can produce huge amounts of coverage data, which can be overwhelming when trying to analyze which design features belong to which coverage points. This is the reason coverage-driven verification has many limitations that affect its usability and scalability.
- **Plan-based/plan-driven MDV** broadens the scope of what is captured and measured to include checks, assertions, software, and time-based data points that are encompassed in the term “metrics.” The plan-based notation means that you can organize your verification plans by features, by milestones, by design hierarchy, or all of the above including your own definitions.

Review of the Metric Driven Verification (MDV) Cycle



11 © Cadence Design Systems, Inc. All rights reserved.

cadence

MDV is a powerful, open and innovative approach to pre-silicon functional verification which Cadence pioneered over 10 years ago, and we continue to be the industry leader in this area. Our plan to closure process is represented by this diagram of planning, construction, execution and measurement and analysis of the results.

On the planning side, we connect to top-level plans, excel tests lists, requirements management systems and specifications. vManager provided a specific purpose verification planning tool which is used for plan authoring. Construction of your testbench is done through UVM for hardware testing, and SLN for software and system testing. This is where you define what metrics you wish to capture that are representative of the features you are trying to verify.

Next is the execution side. The vManager tool is a multi-user and multi-engine environment able to support the various Cadence verification engines for optimal productivity and performance. From Xcelium simulation and Jasper formal for IP-based designs to Palladium® with software-driven testing for a subsystem or SOC designs, the MDV solution is very scalable and flexible to accommodate digital and analog, hardware and software, IP to SOC. Measurement and analysis are done by the vManager tool, which allows many ways to drill into the results to identify holes and display the results. vManager uses a commercial SQL database to aggregate data such that real-time trends and web charts are visible and available to all. Reporting is done through text or as show, HTML reports, which are drillable down to the lowest levels of the design.

- The figure shows the complete MDV cycle starting from planning to the signoff phase.
- The vManager tool spans the planning, execution of the tests/plan, measurement of metrics and finally the analysis and management of the available metrics.
- The construction phase basically includes building the verification infrastructure such as test cases, testbench, assertions, etc., based on the UVM methodology.

Benefits of Metric Driven Verification

Higher Quality

- Find more bugs faster, combining formal ABV, constrained random simulation, and HW/SW emulation.
- Information on incomplete areas.

Better Productivity

- Automation for many everyday tasks.
 - Coverage and failure analysis, running regressions, reporting.
- Combine metrics across verification approaches to reduce effort duplication.
- Optimize human and compute resource utilization.

Improved Predictability

- Dynamic visibility into verification closure with respect to the plan.
- Ability to measure progress based on project-specific milestones.
- Answers the question – “*are we done yet?*”

12 © Cadence Design Systems, Inc. All rights reserved.



Hence MDV answers the top business concerns:

1. Schedule to DV completion
2. Cost of Tapeout (verification effort/people)
3. Functional bug(s!) causing re-spin or recall

Why Is Planning Important?

Any process that needs clear and measurable goals and verification is no exception and definitely needs planning!

- You can gauge improvement only by what you can clearly measure.
- Failing to capture goals at the beginning of a project means no definition of what needs to be done.



MDV defines a verification plan – a list of “what to verify” or the “features” of the design intention.

The traditional test plan is a list of “how to verify” the methods of a design.

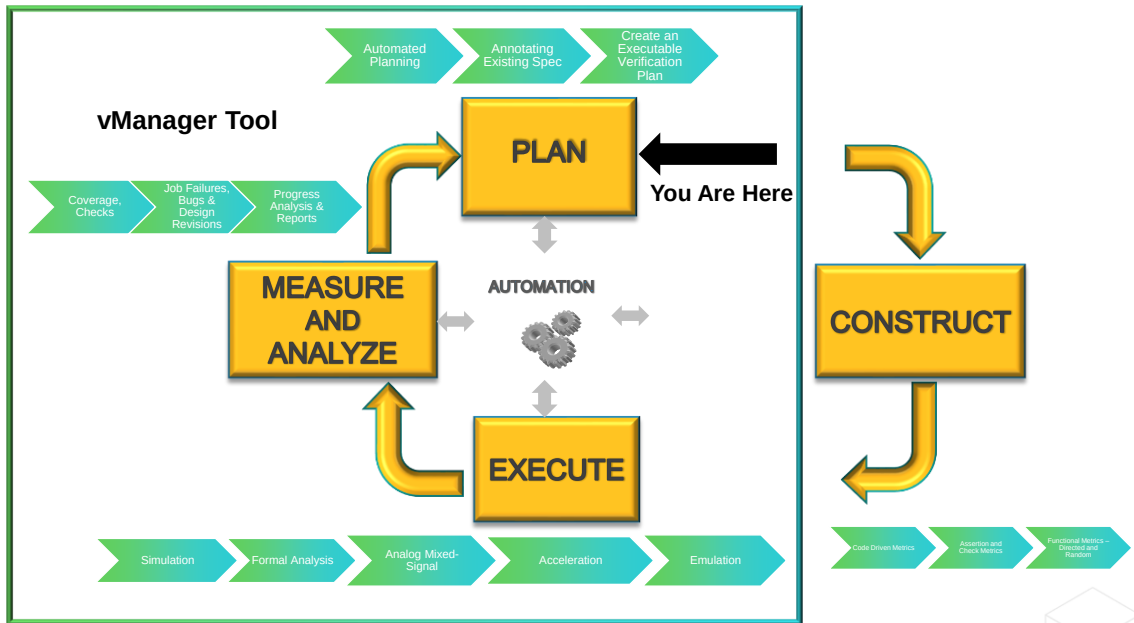
Verification plan – closed and executable process – is measurable.

Test plan is an open process and not executable.

The MDV vManager™ flow starts with a verification plan.

This page does not contain notes.

Planning Phase



14

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

Motivation for Verification Planning

Given limited resources, we would like to:

- Maximize quality and quantity of verification achieved.
- Mitigate project risk.
- Efficiently execute the task of implementing the verification environment:
 - Understand requirements before writing a bunch of code.
- Forecast and track verification resource requirements.
- Achieve transparency of verification progress.
- Give early warning of missed schedules and design/verification issues.
- Assure an effective plan is produced.
- Measure progress relative to the plan throughout the project.



This page does not contain notes.

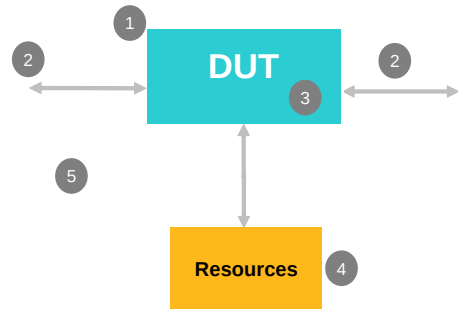
What Is the Feature Planning Approach?

Feature planning focuses on “what” to verify versus “how.”

- From specifications.
- From experience of the project team.

Some examples of design features include:

1. Device mode and configuration options. Traffic or protocol handling.
2. Protocol or device exception handling.
3. Internal device properties.
4. Arbitration for external resources.
5. Typical and critical use cases.



This page does not contain notes.

Feature-Based Design Analysis

Prerequisite: Design intent is understood by the stakeholders.

- A stakeholder is a participant in a verification project, such as Design/Verification Engineer, Program Manager, etc.

Review the design specifications and consider the following:

- What functionality does the design provide?
- What expected behavior is triggered by a scenario? Or a test case?
- Do not brainstorm as to how the features should be verified.
- Write down a description for each feature. Use words like “The design...”
 - ...supports...”
 - ...is able to...”
 - ...handles...”



This page does not contain notes.

Feature-Based Design Analysis (continued)

- Avoid phrases (because they have a “Directed Test” implication) like:
 - “check that...”
 - “verify...”
 - “if <this> then <that>”
- Organize the identified features in a hierarchical fashion:
 - Structure should consider the need for tracking progress.
- Granularity and fidelity of identified features depend on:
 - Stakeholders involved.
 - Design complexity.
- Create separate categories for:
 - Architectural details.
 - Implementation details.
 - Specialized scenarios (next slide).



This page does not contain notes.

Feature Planning (*What to Verify?*)

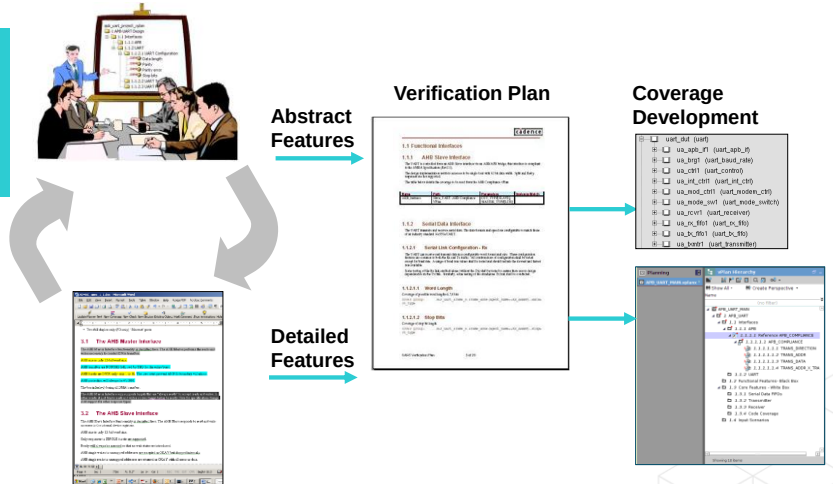
- A verification plan is needed to capture the features that need to be verified.
- Peer review sessions, as well as a thorough spec analysis, are good techniques for extracting the main features and structure of the verification plan.
- Spec annotation or peer review. Start with one and continue to the other...

Peer Review, Brainstorming

Identify things that were either not in the spec or are not explicit in the spec (like the relationship between attributes – cross), get alignment between different stakeholders about spec interpretation, etc.

Spec Annotation

It makes sure you did not forget critical features that need to be verified from the spec. It is unlikely that a peer review will uncover every feature.



Example of Feature-Based Plan: The “WHAT”

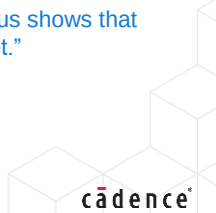
Focuses First on “What,” Not “How,” to Allow Involvement of the Entire Team

Feature-Based Plan

1. System Bus Subsystem features to be verified:
 - 1.1 **Transfer Types:** The bus supports both single transfers and burst transfers.
 - 1.2 **Transfer Directions:** The bus supports both read and write transfers.
 - 1.3 **Transfer Targets:** The bus supports transfers from the master to all slaves.
2. Packet Interface
 - 2.1 ...
3. Graphics Engine
 - 2.2 ...

Benefits

- Plan-specific measurable goals for what must be verified.
 - For example, it can be measured with functional coverage.
- Plan expresses the exact intent of “what” will be verified (not “how”) – can be understood by **all** stakeholders.
 - For example, the System Architect who doesn’t know SV syntax.
- Plan provides tracking of exactly what has been verified.
 - “Your latest plan-based coverage status shows that you have not tested burst transfers yet.”



Example of a Feature-Based Plan: The “WHAT” Focuses First on “What,” Not “How,” to Allow Involvement of the Entire Team.

Attribute Elaboration: The “How”

Translating Feature Requirements into Concrete Metric Goals

The verification plan features are translated into a set of quantifiable metrics by asking **HOW** they will be measured.

Which attributes and values are important for each feature?

Where should each value be observed?

When are the values valid to be sampled?

Produce results that can be measured.

Feature-Based Plan

WHAT?

1. System Bus Subsystem features to be verified:
 - 1.1 Transfer Types: The bus supports both single transfers and burst transfers.
 - 1.2 Transfer Directions: The bus supports both read and write transfers.
 - 1.3 Transfer Targets: The bus supports transfers from the master to all slaves.
2. Packet Interface
 - 2.1 ...
 - 2.2 ...
3. Graphics Engine
 - 2.1 ...
 - 2.2 ...

Implementation Guidelines

HOW?

Coverage Items

transfer_types: single, burst
transfer_direction: read, write
transfer_targets: (all slaves) 0xa000, 0xa004, 0xa008, 0xa00C
cross: transfer_types, transfer_direction, transfer_targets

Observed by the system bus monitor

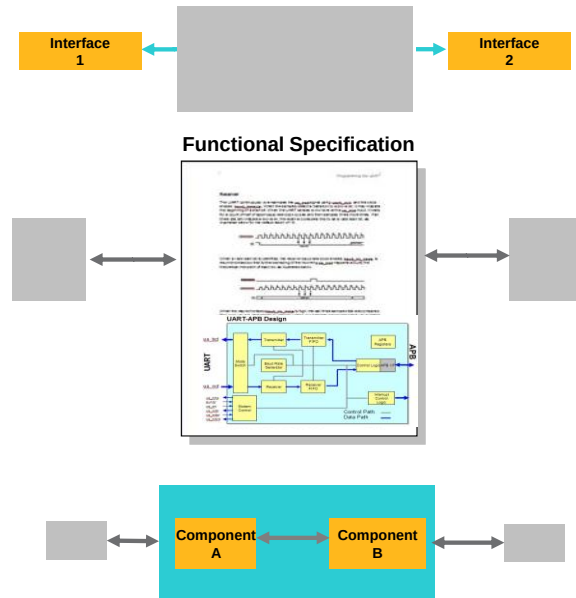
Sampled when transfer_completes

The slide clearly shows the difference between the verification plan and the test plan.

The vPlan or the verification plan describes the “what” features to be verified. The test plan describes “how” these features have to be verified.

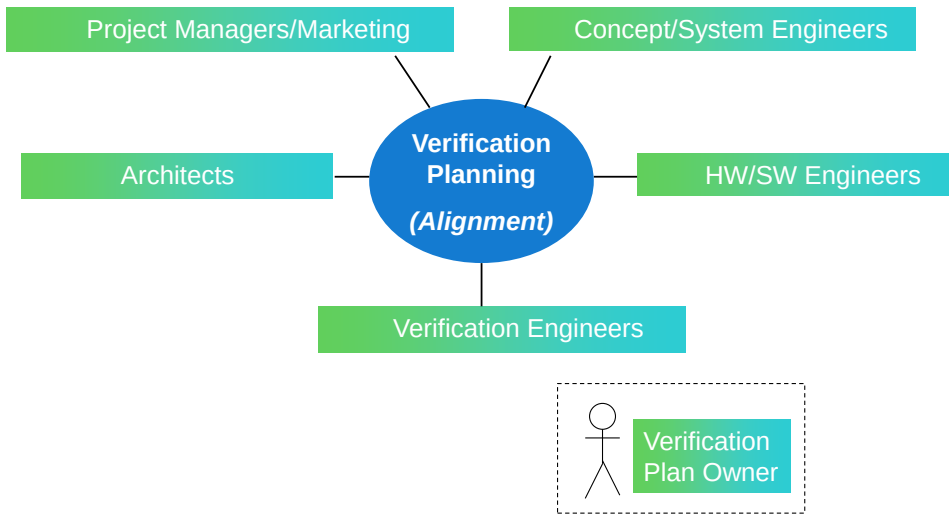
vPlan Template

- Interfaces
 - <interface name>
- Functional Features – Black Box (*features measured outside of the design*)
 - Legal Behavior
 - <feature>
 - Exception Handling
 - <feature>
- Core Features – White Box (*features measured in the design*)
 - Components
 - <component name>
 - Interfaces
 - <interfaces>
- Input Scenarios (*results from test cases*)



The vPlan for the UART_APB design is shown.

Verification Planning Stakeholders



24

© Cadence Design Systems, Inc. All rights reserved.

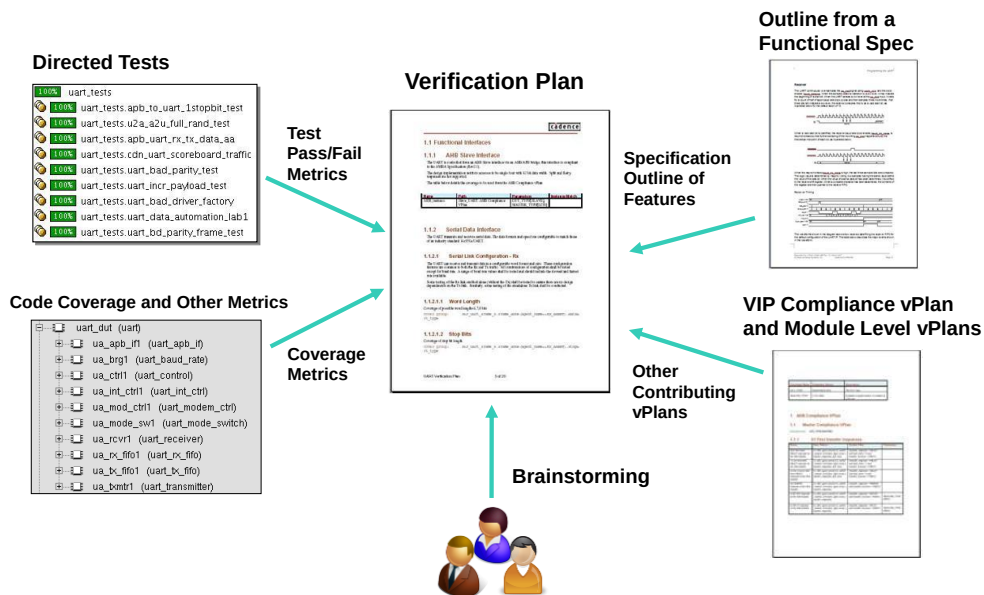


These are all the stakeholders that should be involved in planning for simulation-based verification.

Each of these stakeholders is important, first, for the perspective they can offer to a holistic understanding of the device under verification as described in the module on Verification Planning. Remember that design intent is translated differently by each stakeholder, and the planning session is our opportunity to align these translations.

In addition to contributing their unique viewpoint to the team, each contributor needs to specify and discuss which aspects of each feature are important to them. In some cases, they will help to define metrics that will objectively measure verification closure vs. their concerns. In other cases, they will note metrics that have already been defined and add them to their individually defined “view” of the verification plan.

Verification Planning: Many Sources



25 © Cadence Design Systems, Inc. All rights reserved.

There are many sources that constitute the final vPlan and make it a complete one.

The Top-Down Approach

The top-down approach is a generalized brainstorming approach that works for any product regardless of whether written product specifications exist. In this approach, using a block diagram of the design for reference, you list two categories of features – product requirements and design requirements. In each of these categories, you list interface and core requirements. As the requirements are defined, a team member enters them into the vPlanner's verification plan (vPlan) editor.

The Bottom-Up Approach

The bottom-up approach is a more systematic approach that focuses on the specifications for the current project. The team reviews the specifications paragraph by paragraph and identifies the features that are described. Using vPlanner, a team member selects the appropriate text in the specifications document and creates a new section or another element in the verification plan. The selected text appears in the verification plan as part of the description of the new element.

The bottom-up approach is more systematic, but it creates a verification plan whose structure reflects the functional requirements. As a result, the plan might not be easily reusable by other projects.

For best results, use a combination of the two approaches. For example, brainstorm key features and then annotate the specification to ensure nothing is missing from the plan.

Verification Planning: Main Challenges

- How do I know whether my plan is complete?
 - No missing pieces
 - Up-to-date
 - Reflects functional and design specifications including spec changes
- How do we write a plan that can be reused later?
 - From module to system
 - Between projects
 - Among different owners in the team
- How do we track our progress against this plan?
 - Dynamically track completeness of each verification plan feature using measured verification metrics (coverage driven and assertion-based)
 - Accurately predict verification project schedule and track progress *versus* goals established for each project milestone



This page does not contain notes.

Human Factors

- Stakeholders who come up with or influence the feature-based plan are humans.
- It is in human nature to make mistakes!
- These mistakes should be accounted for upfront while planning as much as possible.
- Changes/modifications happen inevitably during the course of projects because of this.
- Verification plan should be flexible and mature enough to include these changes on the fly.
- MDV methodology mainly automates most of these processes.



This page does not contain notes.